

Ain Shams University
Faculty of Engineering
Electronics and Communications Engineering
Department

Digital Design Verification
Final Project Report

SPI Master Controller Verification

UVM-Based Verification Environment
for an APB-Slave SPI Master IP

Spring 2026

Hesham Nabil Hosny Maher	2101109
Mahmoud Hesham Abdelmoniem	2100613
Ahmed Hamdy Elhusseiny	2101133
Mohamed Ibrahim Elmonier	2101599
Ahmed Mohamed Afifi	2100536
Moamen Ali Gad	2101225
Kareem Mousa Mahmoud	2100295

Contents

1 Abstract	2
2 DUT Overview	2
3 Register Map	2
4 Verification Methodology	3
4.1 UVM Testbench Architecture	3
4.2 Top-Level Testbench	4
4.3 APB Agent	4
4.4 SPI Agent	4
4.5 Reference Model	5
4.6 Scoreboard	5
5 Sequence Library	5
6 Test Cases	5
6.1 Base Test	5
6.2 Sanity Test	6
6.3 Full Requirement Test	6
7 Requirement-to-Test Mapping	6
8 Assertions	7
8.1 APB Assertions	7
8.2 SPI Core Assertions	7
9 Functional Coverage	7
9.1 Coverage Model	7
10 Coverage Results	8
10.1 Coverage Discussion	8
11 Regression Results	9
12 Results and Discussion	9
13 Challenges	9
14 Conclusion	10

1 Abstract

This report documents the complete Universal Verification Methodology (UVM) verification environment developed for the SPI Master Controller final project. The design under verification is an APB-slave SPI Master IP supporting 8-bit, 16-bit, and 32-bit transfers, all four SPI modes, programmable clock division, four active-low slave-select outputs, loopback operation, TX/RX FIFOs, programmable inter-transfer delay, and interrupt generation.

The verification environment was built using UVM and includes APB and SPI agents, drivers, monitors, sequence items, reusable requirement-oriented sequences, a reference model, scoreboard, functional coverage collector, and SystemVerilog assertions. The verification plan maps the specification requirements R1–R25 to directed and reusable stimulus, output checking, and coverage goals.

2 DUT Overview

The Design Under Test (DUT) is a synthesizable SPI Master Controller with a 32-bit APB slave interface for configuration, control, and data management. The controller bridges the processor-side APB bus with external SPI devices while providing programmable communication, status monitoring, and interrupt generation capabilities.

The design consists of two primary RTL blocks: `apb_regfile`, which manages register access, FIFOs, configuration, and interrupt handling, and `spi_core`, which implements SPI clock generation, shift operations, transfer control, loopback functionality, and delay insertion.

The SPI Master supports all four SPI operating modes, programmable transfer widths (8, 16, and 32 bits), MSB-first and LSB-first operation, dedicated TX/RX FIFOs, programmable clock division, four active-low slave-select outputs, loopback mode, programmable inter-transfer delay, and masked sticky interrupt generation.

3 Register Map

Table 1: SPI Master Register Map

Offset	Name	Access	Description
0x00	CTRL	R/W	Control register: enable, master mode, SPI mode, bit ordering, loopback, and width.
0x04	STATUS	RO	Busy, FIFO status, and overflow status flags.
0x08	TX_DATA	WO	TX FIFO push port.
0x0C	RX_DATA	RO	RX FIFO pop port.
0x10	CLK_DIV	R/W	SPI clock divider.
0x14	SS_CTRL	R/W	Slave-select enable and value control.
0x18	INT_EN	R/W	Interrupt enable mask.
0x1C	INT_STAT	R/W1C	Sticky interrupt status register.
0x20	DELAY	R/W	Inter-transfer delay in SCLK half-cycles.

4 Verification Methodology

The verification environment was developed using Universal Verification Methodology (UVM) to provide a modular and reusable framework for validating the SPI Master Controller. The environment follows a layered architecture where sequences generate APB and SPI transactions, drivers stimulate the DUT, monitors capture interface activity, and a scoreboard compares DUT outputs against a reference model.

4.1 UVM Testbench Architecture

The testbench is built around a top-level UVM environment integrating dedicated APB and SPI agents. The APB agent handles register access transactions, while the SPI agent monitors serial communication behavior and protocol timing.

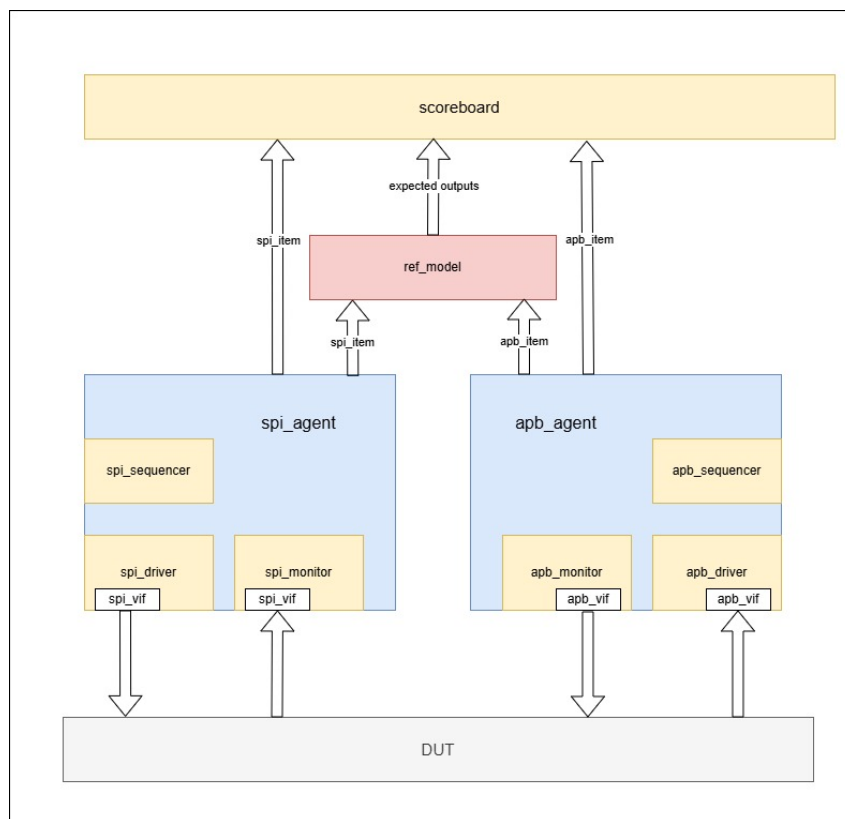


Figure 1: Implemented UVM Verification Environment Hierarchy

Figure 1 The hierarchical organization of the implemented UVM verification environment.

Each agent contains the standard UVM building blocks including sequencer, driver, and monitor components connected to their corresponding virtual interfaces. The APB and SPI monitors forward collected transactions toward both the reference model and scoreboard, enabling automated comparison between DUT behavior and expected functionality.

The reference model generates expected outputs based on observed transactions and DUT configuration, while the scoreboard performs end-to-end validation by comparing predicted and observed results. This layered architecture promotes modularity, reusability, and scalability while enabling comprehensive verification of the SPI Master Controller.

Functional correctness is verified through a behavioral reference model and scoreboard infrastructure, enabling automated checking of register operations, FIFO functionality, interrupt handling, and SPI transfer behavior. Functional coverage collection is incorporated to measure verification completeness, while SystemVerilog assertions are bound to the DUT hierarchy to validate protocol rules and critical design properties.

The environment is integrated through `tb_top.sv`, which instantiates the DUT, interfaces, clock and reset generation logic, and launches the UVM test flow.

4.2 Top-Level Testbench

The top-level testbench instantiates the APB and SPI interfaces, connects them to the DUT, starts UVM using `run_test()`, and binds the assertion modules into the DUT hierarchy. This allows the verification environment to observe both external protocol behavior and internal design states needed for assertions and coverage.

4.3 APB Agent

The APB agent is responsible for generating and monitoring register transactions over the APB interface. It consists of four primary components:

- APB sequence item
- APB driver
- APB monitor
- APB configuration object

The APB driver implements the APB protocol transaction flow by executing the SETUP and ACCESS phases of each transfer. The monitor observes bus activity and forwards collected transactions to the scoreboard and functional coverage collector for checking and coverage sampling.

4.4 SPI Agent

The SPI agent provides monitoring and protocol-level observation of SPI communication behavior. Similar to the APB agent, it is composed of:

- SPI sequence item
- SPI driver
- SPI monitor
- SPI configuration object

The SPI monitor reconstructs serial transfers from `SCLK`, `MOSI`, `MISO`, and `SS_n` signals, enabling verification of transfer ordering, timing behavior, SPI operating modes, and slave-select functionality.

4.5 Reference Model

The reference model provides cycle-independent prediction of expected DUT behavior and serves as the golden model for functional checking. It tracks internal design state including register contents, FIFO behavior, interrupt status generation, loopback operation, and expected receive-path data.

4.6 Scoreboard

The scoreboard performs automatic comparison between DUT outputs and reference model predictions. It verifies register behavior, FIFO ordering, interrupt functionality, receive data correctness, and protocol-visible DUT responses. Any mismatch between predicted and observed behavior is reported through the standardized verification error-reporting mechanism.

5 Sequence Library

The sequence library was organized around the numbered requirements in the specification. This makes the verification plan traceable from requirement to stimulus and checker.

Table 2: Main Sequence Files

Sequence File	Purpose
<code>sequence_base.sv</code>	Base class for common APB/SPI sequence utilities.
<code>apb_reset_seq.sv</code>	Applies and checks APB-side reset behavior.
<code>spi_reset_seq.sv</code>	Applies and checks SPI-side reset behavior.
<code>r1_r2_reg_seq.sv</code>	Verifies register read/write behavior and reset values.
<code>r3_ctrl_en_seq.sv</code>	Verifies CTRL.EN disabled-state behavior and FIFO flush.
<code>r4_r8_spi_protocol_seq.sv</code>	Verifies SPI modes, width, ordering, and SCLK divider behavior.
<code>r9_r15_fifo_seq.sv</code>	Verifies TX/RX FIFO ordering, depth, and overflow/empty behavior.
<code>r16_r18_irq_seq.sv</code>	Verifies interrupt masking, W1C behavior, and W1C race priority.
<code>r19_r23_misc_seq.sv</code>	Verifies loopback, SS control, delay, APB response, and reserved offsets.
<code>main_sequence.sv</code>	Runs the complete sequence set.

6 Test Cases

6.1 Base Test

The `spi_test_base.sv` file defines the common UVM test infrastructure. It creates the environment, configures virtual interfaces, and provides common functionality used by derived tests.

6.2 Sanity Test

The `sanity_test.sv` file performs a basic bring-up test. It verifies that the environment can reset the DUT, configure basic registers, perform a simple transfer, and complete without protocol or scoreboard errors.

6.3 Full Requirement Test

The `full_req_test.sv` file runs the full requirement-oriented sequence set. It targets requirements R1–R25 and is intended as the main regression test for functional closure.

7 Requirement-to-Test Mapping

The verification plan was organized around the specification requirements R1–R25. Instead of treating the requirements as isolated checks, related requirements were grouped into focused verification areas. Each group is covered by a dedicated sequence and checked using the scoreboard, reference model, functional coverage, and assertions where applicable.

Table 3: Grouped Requirement-to-Test Mapping

Req.	Verification Focus	Main Sequence / Checker
R1–R2	Register read/write behavior and reset-value verification.	<code>r1_r2_reg_seq</code> ; APB monitor, scoreboard, and register coverage.
R3	Disabled-state behavior, FIFO flush, idle SCLK, and slave-select reset behavior.	<code>r3_ctrl_en_seq</code> ; scoreboard and control-state assertions.
R4–R8	SPI mode behavior, MOSI timing, bit ordering, transfer width, and SCLK divider operation.	<code>r4_r8_spi_protocol_seq</code> ; SPI monitor, protocol assertions, and mode/width coverage.
R9–R15	TX/RX FIFO ordering, FIFO depth, full/empty behavior, overflow handling, and empty RX reads.	<code>r9_r15_fifo_seq</code> ; reference model, scoreboard, and FIFO coverage.
R16–R18	Interrupt masking, sticky status bits, W1C clearing, and W1C race priority.	<code>r16_r18_irq_seq</code> ; interrupt scoreboard checks and IRQ assertions.
R19–R23	Loopback mode, slave-select equation, delay insertion, APB response behavior, and reserved offsets.	<code>r19_r23_misc_seq</code> ; scoreboard, SPI monitor, and miscellaneous coverage bins.
R24–R25	Divider-zero operation and mid-transfer configuration stability.	Covered inside protocol-focused sequences using SCLK checks and sampled-configuration assertions.

8 Assertions

SystemVerilog Assertions (SVA) were integrated into the verification environment to validate protocol compliance and critical internal design behavior. Assertions were bound externally to the DUT hierarchy, allowing verification checks to be added without modifying the RTL implementation.

8.1 APB Assertions

The APB assertion set focuses on interface correctness and register-level behavior. These assertions verify transaction sequencing requirements, signal stability across APB phases, and protocol constraints such as zero wait-state operation and constant error response behavior. Additional checks validate interrupt generation logic, overflow status handling, and reset-state correctness for key control and status signals.

8.2 SPI Core Assertions

The SPI assertion set targets functional correctness of the serial communication engine. These properties verify transfer control behavior, MOSI timing relationships, loopback functionality, SCLK idle-state restoration, receive-path operation, delay insertion behavior, and transfer-width consistency. Assertions are also used to validate internal control logic, ensuring stable operation across all supported SPI configurations.

9 Functional Coverage

Functional coverage was implemented to measure verification completeness and ensure that all major specification features were exercised during simulation. Coverage collection was integrated into the UVM environment and sampled APB transactions, SPI protocol configurations, FIFO activity, interrupt generation behavior, loopback operation, and configuration corner cases.

9.1 Coverage Model

The coverage model was structured to target the primary verification dimensions of the SPI Master Controller, including:

- SPI operating modes, transfer widths, and bit-order configurations
- FIFO operation, occupancy levels, and overflow scenarios
- Interrupt generation, masking, and clearing behavior
- Register access patterns and reset-state verification
- Clock divider configurations and delay insertion functionality

Cross coverage was implemented between key protocol parameters to ensure that supported operating configurations were validated in combination rather than in isolation. Particular emphasis was placed on protocol corner cases and boundary conditions to improve verification confidence and contribute toward coverage closure.

10 Coverage Results

Coverage closure was achieved through a combination of directed verification sequences, functional coverage collection, assertion-based verification, and scoreboard checking. The generated coverage reports indicate strong verification completeness across the major RTL blocks and verification components.

Table 4: Coverage Summary

Coverage Metric	Result
Register File Assertion Coverage	100.00%
Register File Branch Coverage	100.00%
Register File Condition Coverage	100.00%
Register File Expression Coverage	92.85%
Register File Statement Coverage	100.00%
Core Assertion Coverage	100.00%
Core Branch Coverage	96.49%
Core Condition Coverage	89.47%
Core Expression Coverage	100.00%
Core FSM Coverage	100.00%
Core Statement Coverage	98.70%
Top-Level Statement Coverage	100.00%
Functional Covergroup Coverage	100.00%
DUT Toggle Coverage	98.69%
Final Functional Coverage	100.00%

10.1 Coverage Discussion

The achieved results demonstrate strong verification completeness across both functional and structural coverage metrics. Assertion coverage reached 100% for both the register file block and SPI core block, confirming that all implemented verification properties were exercised successfully during simulation.

The register file block achieved full branch, condition, and statement coverage, indicating complete validation of control paths and register-level functionality. The SPI core similarly demonstrated high structural coverage, including full FSM state and transition coverage, validating operation across all supported protocol configurations.

Functional coverage reached 100%, indicating successful stimulation of all planned verification scenarios defined in the coverage model. Remaining uncovered structural bins are primarily associated with intentionally static signals, reserved fields, and design behavior that is not expected to toggle during normal operation.

11 Regression Results

Regression execution was performed using the complete UVM verification environment together with automated checking through the scoreboard and assertion infrastructure.

Table 5: Regression Summary

Metric	Result
Total Transactions	31907
Correct Checks	31907
Error Count	0
Pass Rate	100.00%
Final Test Status	PASSED

The regression completed successfully without scoreboard mismatches or assertion failures. All observed transactions matched the expected behavior predicted by the reference model, demonstrating functional correctness of the verified SPI Master implementation.

12 Results and Discussion

The verification environment successfully targets all numbered requirements R1–R25 using a requirement-oriented sequence library. The use of a reference model and scoreboard enables automatic checking of register behavior, FIFO ordering, RX data, interrupt status, and APB/SPI protocol behavior.

The assertion set complements the scoreboard by checking protocol timing and internal design invariants. This combination improves bug-detection capability because some bugs are best detected through end-to-end data comparison, while others are more naturally detected through local temporal properties.

13 Challenges

The main verification challenges were:

- Correctly modeling FIFO behavior and overflow conditions.
- Handling interrupt sticky status bits and W1C clearing behavior.
- Verifying the W1C race condition where a hardware event and software clear occur together.
- Checking SPI timing across all CPOL/CPHA modes.
- Ensuring that mid-transfer updates to configuration registers do not affect the active transfer.
- Keeping the regression efficient while still covering all functional requirements.

14 Conclusion

A complete UVM verification environment was developed for the APB-slave SPI Master Controller. The environment includes reusable UVM agents, drivers, monitors, sequence items, sequences, a reference model, scoreboard, coverage collector, and SystemVerilog assertions.

The verification plan maps each specification requirement to directed stimulus, checking, and coverage. The coverage report demonstrates strong assertion and code coverage, with APB and SPI core assertion coverage reaching 100%. The environment is structured to support regression execution, coverage collection, and grader-compatible bug detection.